



《AI大模型Ragent项目》——知识问答在后端经历了哪八个阶段？

来自：拿个offer-开源&项目实战



马丁

2026年04月20日 22:03

开篇引言

假设你在一家电商公司做开发，公司上了一套智能客服助手，接入了 3C 数码、家电、服装等多个品类的商品知识库，还对接了订单系统、物流系统等业务接口。某天，一个用户在对话框里输入了一句话：

iPhone 16 Pro 的退货政策是什么？

然后点了发送。

几秒后，答案像打字机一样一个字一个字蹦出来，不仅引用了退货政策文档，还能实时查到这个用户的订单状态。看起来很丝滑，但这背后到底发生了什么？

从用户按下回车到最后一个 Token 推送完毕，后端至少要做这些事：加载这个用户之前聊了什么、把问题改写成更适合检索的形式、判断应该去哪个知识库找答案、从向量数据库捞文档、把文档和问题拼成 Prompt、调大模型生成答案、再一个 Token 一个 Token 通过 SSE 推给前端……

这些环节按什么顺序跑？哪些可以跳过？如果用户只是打个招呼说了句你好，还需要去知识库检索吗？如果检索什么都没找到，要不要硬着头皮让模型编一个答案？

这就是本篇要回答的问题。

在 Ragent 项目中，整个问答流程由一个叫 StreamChatPipeline 的类编排，它把上面这些事情拆成了八个阶段，按固定顺序依次执行，中间设置了三个短路点——满足特定条件时提前结束，不用走完全部八步。

本篇是 AI 知识问答系列的第 1 篇，预计共 18 篇。它是整个系列的全景地图，不深入任何一个环节的代码细节，目标只有一个：看完之后，你脑子里有一张完整的地图，后续每篇文章都能在这张地图上找到自己的位置。

请求进入 Pipeline 之前

用户的问题从浏览器出发，到达 StreamChatPipeline 之前，还要经过两层防护和一次初始化。这三步不属于八个阶段，但它们决定了请求能不能进入 Pipeline。

1. 幂等提交拦截

Controller 层的 chat() 方法上标注了 @IdempotentSubmit 注解，基于用户 ID 加分布式锁。用户快速连点两次发送按钮，第二次请求会被直接拦截，返回“当前会话处理中，请稍后再发起新的对话”。

当然，大家可以根据实际要求拦截，这里仅是其中一种拦截策略。

这是一个纯工程手段，和 RAG 本身没有关系，但在生产环境中非常必要——没有它，同一个用户的两次请求会同时进入 Pipeline，导致记忆写入冲突、资源浪费。

2. 队列式并发限流

Service 层的 streamChat() 方法标注了 @ChatRateLimit，由 ChatRateLimitAspect 拦截，内部委托给 ChatQueueLimiter 处理。

打个比方，大模型推理就像餐厅的后厨，灶台数量有限。ChatQueueLimiter 用 Redis 信号量控制同时有多少个请求可以在后厨炒菜（并发坑位），超出的请求排队等待。如果等太久超过了最大等待时间，直接返回排队超时，不让用户干等。

队列式并发限流是 Ragent 的一个核心工程设计，后面会专门展开讲。

3. 全链路 Trace 初始化

请求拿到并发坑位后，在进入 Pipeline 之前，AOP 切面会初始化全链路追踪上下文——生成 traceId 和 taskId，通过 RagTraceContext（基于 TransmittableThreadLocal，支持跨线程池传递）贯穿后续所有阶段。

每个阶段的耗时、输入输出都会记录到 Trace，出了问题可以精确定位卡在哪一步。这是生产环境排查性能瓶颈的基础设施。

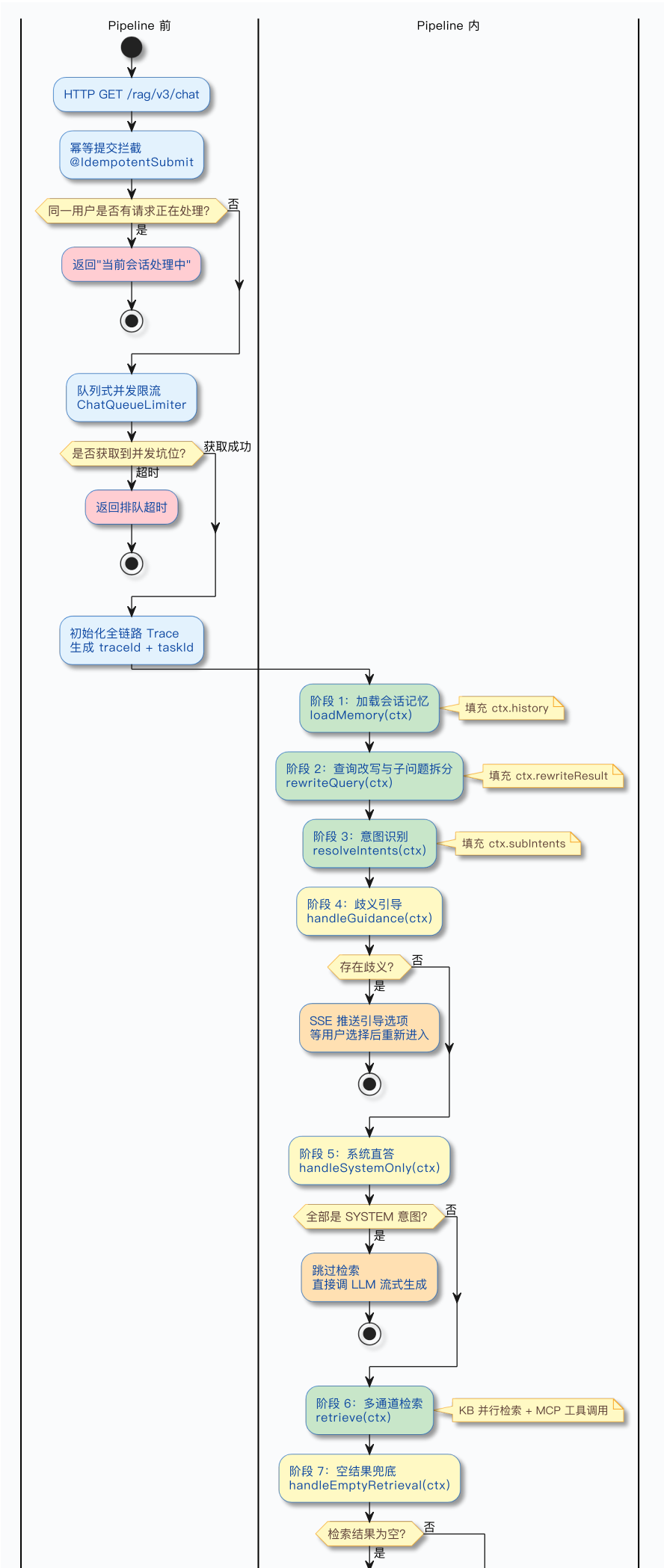
八个阶段全景图

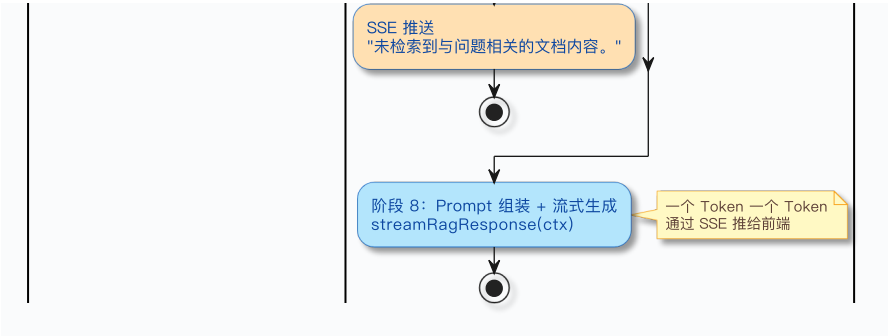
整个链路从 HTTP 请求到流式响应，完整路径长这样：

```
HTTP GET /rag/v3/chat
└─ RAGChatController.chat()
```

```
@IdempotentSubmit (防重复提交)
└─ RAGChatServiceImpl.streamChat()
    └─ @ChatRateLimit → ChatRateLimitAspect
        └─ ChatQueueLimiter.enqueue() (排队等信号量)
            └─ invokeWithTrace() (初始化全链路 Trace)
                └─ StreamChatPipeline.execute(ctx)
                    阶段 1: loadMemory      - 加载会话记忆
                    阶段 2: rewriteQuery   - 查询改写与子问题拆分
                    阶段 3: resolveIntents - 意图识别
                    阶段 4: handleGuidance - 歧义引导 [短路点 #1]
                    阶段 5: handleSystemOnly - 系统直答 [短路点 #2]
                    阶段 6: retrieve       - 多通道检索 (KB + MCP)
                    阶段 7: handleEmptyRetrieval - 空结果兜底 [短路点 #3]
                    阶段 8: streamRagResponse - Prompt 组装 + 流式生成
```

下面这张 PlantUML 活动图，把 Pipeline 前的防护和 Pipeline 内的八个阶段、三个短路分支都画了出来：





而这张地图在代码中的体现，就是 StreamChatPipeline.execute() 方法，只有 20 行，却是整个问答系统的骨架：

```
public void execute(StreamChatContext ctx) {
    loadMemory(ctx);           // 阶段 1
    rewriteQuery(ctx);          // 阶段 2
    resolveIntents(ctx);        // 阶段 3

    if (handleGuidance(ctx)) { // 阶段 4 - 短路点 #1
        return;
    }
    if (handleSystemOnly(ctx)) { // 阶段 5 - 短路点 #2
        return;
    }

    RetrievalContext retrievalCtx = retrieve(ctx); // 阶段 6
    if (handleEmptyRetrieval(ctx, retrievalCtx)) { // 阶段 7 - 短路点 #3
        return;
    }

    streamRagResponse(ctx, retrievalCtx); // 阶段 8
}
```

每个阶段是一个私有方法，通过 boolean 返回值实现短路控制——返回 true 表示当前阶段已经处理完毕，Pipeline 直接 return，不再往下走。

这种设计的好处是一目了然：你看这 20 行代码，就知道一次问答走了哪些步骤、在哪里可能提前结束。后续每篇文章要讲的内容，都对应这里的某一个方法调用。

阶段 1：加载会话记忆

用一句话概括：把用户当前消息追加到记忆存储，然后加载完整的对话历史。

为什么这是第一步？因为后续的查询改写和意图识别都需要对话上下文。比如用户上一轮问了 iPhone 16 Pro 的价格，这一轮追问“那它能退货吗”——没有对话历史，改写模块根本不知道“它”指的是什么。

这个阶段做完后，ctx.history 被填充为一个 List<ChatMessage>，包含了这个会话的完整对话记录（当然，实际生产中会有滑动窗口或摘要压缩来控制长度，不会无限膨胀）。

阶段 2：查询改写与子问题拆分

用一句话概括：把用户的原始问题改写成更适合检索的形式，同时把复合问题拆成多个子问题。

这个阶段有两个输出：

改写后的完整问题 (rewrittenQuestion)：消除代词引用、补全上下文信息。比如用户说“那它的保修期呢”，结合对话历史改写为“iPhone 16 Pro 的保修期是多少”。

子问题列表 (subQuestions)：如果用户问了一个复合问题，会被拆成多个独立的子问题。比如“iPhone 和 AirPods 的退货政策分别是什么”，拆成“iPhone 的退货政策是什么”和“AirPods 的退货政策是什么”两个子问题。

为什么放在这个位置？往前看，它需要阶段 1 的对话历史才能正确消解代词；往后看，阶段 3 的意图识别需要针对每个子问题分别做分类。

阶段 3：意图识别

用一句话概括：对每个子问题并行做意图分类，判断应该去哪个知识库、调哪个工具、还是系统直接回答。

Ragent 的意图体系是一棵树，每个叶子节点代表一个具体的意图，有三种类型：

意图类型	含义	举例
KB	知识库检索	去 3C 数码知识库查退货政策
MCP	工具调用	调订单系统接口查物流状态
SYSTEM	系统直答	用户说你好，直接让模型回答

每个子问题会命中若干意图节点，每个节点带一个分数。但命中太多也不行——意图越多，后续检索和调用的开销越大。所以这里有一个封顶算法：保证每个子问题至少保留最高分意图，然后按全局分数排序分配剩余配额，在多样性和性能之间取平衡。

意图树的结构设计、Prompt 打分方案、算法是系统中最复杂的部分之一，第 5~9 篇用 5 篇的篇幅详细展开。

阶段 4：歧义引导 [短路点 #1]

用一句话概括：检测多个意图之间是否存在歧义，如果有就反问用户，让用户自己选。

回到开头的例子。如果用户问的不是“iPhone 16 Pro 的退货政策”，而是只说了一句“退货政策是什么”——3C 数码的知识库和家电的知识库都高分命中了。系统不确定用户问的是哪个品类，猜错了不如不猜。

这时候 handleGuidance 会检测到歧义，通过 SSE 直接推送一段引导文本给前端，类似于“请问您想了解的是 3C 数码还是家电的退货政策？”让用户点选之后，带着明确的选择重新进入 Pipeline。

注意，这个阶段触发短路时，不调用大模型，直接推送引导选项就返回了。这是第一个短路点。

阶段 5：系统直答 [短路点 #2]

用一句话概括：如果所有意图都是 SYSTEM 类型，跳过检索，直接用系统 Prompt 调 LLM 回答。

什么情况下会触发？用户说“你好”、“你能做什么”、“谢谢”这类不需要查任何知识库也不需要调任何工具的问题。阶段 3 把它们全部归类为 SYSTEM 意图，到了阶段 5 检查发现——所有子问题的所有意图节点都是 SYSTEM 类型，那就不需要走检索了。

跳过阶段 6 和 7，直接拿意图节点上配置的 promptTemplate（如果有的话，没有就用默认系统 Prompt）调 LLM 流式生成回答。用户打个招呼，没必要去向量数据库里搜一圈，白白浪费时间和资源。

这是第二个短路点。和阶段 4 不同的是，这里虽然跳过了检索，但还是调用了大模型生成回答。

阶段 6：多通道检索

用一句话概括：KB 意图走向量检索（多通道并行 + 去重 + 精排），MCP 意图走工具调用，两条路同时跑。

到了这一步，说明用户的问题确实需要检索或调用工具才能回答。retrievalEngine.retrieve() 内部会根据意图类型分两条路径：

KB 意图：走 MultiChannelRetrievalEngine，对每个命中的知识库并行发起向量检索，检索结果经过去重、精排（Reranker）等后处理，最终筛选出最相关的几条文档片段。

MCP 意图：走 MCPToolExecutor 链，调用外部系统的 API 获取实时数据（比如查订单系统的物流状态）。

这两条路是并行执行的，最终合并到一个 RetrievalContext 对象里，包含 kbContext（知识库检索结果）和 mcpContext（工具调用结果）。

阶段 7：空结果兜底 [短路点 #3]

用一句话概括：如果知识库和工具都没有返回任何结果，直接告诉用户没找到，而不是让模型自由发挥。

retrievalCtx.isEmpty() 检查 kbContext 和 mcpContext 是否都为空。如果是，推送一条固定消息“未检索到与问题相关的文档内容。”然后 return。

为什么要这样做？如果把空的上下文喂给大模型，模型没有参考资料可用，大概率会基于自己的预训练知识编造一个答案。在电商客服场景下，编造的退货政策或价格信息比没有答案更危险——用户可能信以为真。与其让模型凭空编造，不如坦诚告知没找到。

这是第三个短路点。

阶段 8：Prompt 组装与流式生成

用一句话概括：把检索结果、工具数据、对话历史、系统 Prompt 拼成完整的消息列表，调大模型流式生成答案。

走到这里，说明系统已经有了检索到的文档片段和/或工具返回的数据，可以正式生成答案了。这个阶段做四件事：

- 意图聚合：**调 intentResolver.mergeIntentGroup() 把所有子问题的意图合并为一个 IntentGroup，分离出 KB 意图和 MCP 意图。
- Prompt 组装：**调 promptBuilder.buildStructuredMessages() 把系统 Prompt、对话历史、检索上下文、工具数据按照特定结构拼成最终的消息列表。

3. **参数调整**：根据场景差异设置模型参数——纯 KB 场景 `temperature=0.0`（尽量忠实于检索到的原文），有 MCP 混合场景 `temperature=0.3`（允许模型在工具数据基础上做一定整合和发挥）。
4. **流式生成**：调 `llmService.streamChat()` 发起流式请求，答案一个 Token 一个 Token 通过 SSE 推送给前端。同时把返回的 `StreamCancellationHandle` 注册到 `StreamTaskManager`，用户随时可以点击停止生成。

流水线的数据总线

八个阶段之间怎么传递数据？不是靠方法参数一层层往下传，而是共用一个 `StreamChatContext` 对象。每个阶段从 `Context` 里读取上游的输出，处理完后把自己的结果写回 `Context`。
`Context` 的字段分两类：

```
// 不可变输入（构建时设置，整个 Pipeline 期间不变）
private final String question;           // 用户原始问题
private final String conversationId;     // 会话 ID
private final String taskId;             // 任务 ID（用于 Trace 和取消）
private final boolean deepThinking;      // 是否启用深度思考
private final String userId;             // 用户 ID
private final StreamCallback callback;   // SSE 输出回调

// 流水线中间状态（各阶段逐步填充）
@Setter private List<ChatMessage> history;           // 阶段 1 填充
@Setter private RewriteResult rewriteResult;         // 阶段 2 填充
@Setter private List<SubQuestionIntent> subIntents; // 阶段 3 填充
```

为什么这样设计？
不可变输入用 `final` 修饰，在构建 `Context` 时一次性设置。任何阶段都不会意外修改原始请求参数，天然线程安全。
可变字段只有三个，分别对应前三个阶段的输出。阶段 1 填充 `history`，阶段 2 填充 `rewriteResult`，阶段 3 填充 `subIntents`。后续阶段只读取这些字段，不再修改。
这种设计让阶段之间彻底解耦——每个阶段不需要知道前面的阶段是怎么实现的，只需要从 `Context` 里拿到自己需要的数据就行。增加或调整某个阶段时，不会影响其他阶段的接口。

这种设计学问在多个知名框架底层应用，比如 `MyBatis`、`ShardingSphere` 等。

三个短路点的设计哲学

把三个短路点放在一起看，它们的设计意图各不相同，但都指向同一个目标：在合适的时机提前结束，避免做无用功。

短路点	阶段	触发条件	短路行为	设计意图
#1 歧义引导	阶段 4	多个意图高命中，存在歧义	推送引导选项，不调 LLM	与其猜错不如问清楚
#2 系统直答	阶段 5	全部是 SYSTEM 意图	跳过检索，直接调 LLM	不需要检索就不要检索
#3 空结果兜底	阶段 7	KB 和 MCP 都没有返回结果	推送固定兜底消息	与其让模型编造不如坦诚告知

打个比方，整个 Pipeline 像去医院看病。阶段 3 的意图识别就是分诊台，分完诊之后并不是每个患者都需要做全套检查。有些情况可以提前处理：

- 分诊台不确定你该挂哪科，会先问你“您是想看骨科还是皮肤科”——这是**歧义引导**。
- 你只是来开个常规药，不需要做任何检查，直接去取药窗口——这是**系统直答**。
- 检查做完了什么都没查出来，医生直接告诉你“目前没有发现异常”——这是**空结果兜底**。

不是每次请求都需要走完全部八个阶段。短路机制让系统在能确定答案（或确定无法回答）的时候及时止损，既节省了大模型调用的成本，也缩短了用户的等待时间。

18 篇文章地图

本篇画完了全景地图，后续 17 篇每篇会放大地图上的一个格子，深入讲解那个环节的设计和实现。

篇号	标题	对应阶段	关键词
1	一次知识问答在后端经历了哪八个阶段？	全景图	StreamChatPipeline
2	大模型没有记忆，多轮对话怎么做到不失忆？	阶段 1	记忆存储与加载
3	聊了 50 轮 Token 爆了，记忆该压缩还是该丢？	阶段 1	摘要压缩
4	用户说的话和该搜的词，往往不是同一回事	阶段 2	查询改写
5	四分类撑不住 20 个知识库——为什么要设计意图树？	阶段 3	意图树结构
6	怎么让大模型同时给 30 个意图节点打分？	阶段 3	Prompt 模板
7	三个子问题命中了八个意图，该保留哪几个？	阶段 3	封顶算法
8	用户问了个模糊问题，多个知识库都举手了	阶段 4	歧义引导
9	意图分数出来了，该查哪个库、查多少条？	阶段 3→6	意图到检索的映射
10	一次提问同时查三个知识库——多通道并行检索	阶段 6	检索架构
11	三个通道返回 30 条结果，最终只给模型 5 条	阶段 6	后处理流水线
12	知识库答不了的问题，交给 MCP 工具去查	阶段 6	MCP 触发链路
13	用户只说了一句话，工具需要的参数从哪来？	阶段 6	参数提取
14	检索结果、工具数据、对话历史——最终的 Prompt 怎么拼？	阶段 8	上下文组装
15	答案一个字一个字蹦出来，流式生成的完整链路	阶段 8	流式输出
16	用户点了停止生成，集群里发生了什么？	阶段 8	任务取消
17	这次回答慢了 3 秒，到底卡在哪个环节？	全链路	Trace 追踪
18	10 个人同时提问只有 3 个坑位——队列式并发限流	Pipeline 前	限流排队

小结

回顾一下本篇的核心要点：

- 1. 一次知识问答在后端经历八个阶段，由 StreamChatPipeline 线性编排，execute() 方法 20 行代码就是整个系统的骨架。
- 2. 三个短路点让系统能在合适的时机提前结束——歧义时反问用户、纯闲聊时跳过检索、检索为空时坦诚告知——避免无效的检索和生成。
- 3. StreamChatContext 作为数据总线，不可变输入加可变中间状态的设计，让各阶段通过同一个对象传递数据，彼此解耦。
- 4. Pipeline 之外还有三层基础设施：幂等提交拦截防重复、队列式并发限流控资源、全链路 Trace 追踪保可观测。

下一篇，我们放大阶段 1——大模型没有记忆，多轮对话怎么做到不失忆？聊聊记忆的存储与加载机制。

